

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: «Задача коммивояжёра»

Студентка гр. 4345

Бодарева С.А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать точный и приближенный метод решения задачи коммивояжёра.

Задание.

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

- n – количество городов ($5 \leq n \leq 15$)
- Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j . Если $graph[i][j] = 0$ и $i \neq j$, это означает, что прямого пути между городами нет.

Выходные данные:

- Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.
- Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.
- Если такого пути не существует, вывести "no path".

Sample input

```
5
0 1 13 23 7
12 0 15 18 28
21 29 0 33 28
23 19 34 0 38
5 40 7 39 0
```

Sample output

```
78
0 4 2 3 1 0
```

Вариант 7

Точный метод: динамическое программирование, рекурсивная реализация.

Приближённый алгоритм: АЛШ-1.

Требования перед сдачей: прохождением кода в задании 3.1 на Stepik.
Замечания к варианту 7. АЛШ-1 начинать со стартовой вершины.

Независимо от варианта, при сдаче работы должна быть возможность генерировать матрицу весов (произвольную или симметричную), сохранять её в файл и использовать в качестве входных данных.

Основные теоретические положения.

Задача коммивояжёра — одна из самых известных задач комбинаторной оптимизации, заключающаяся в поиске самого выгодного маршрута, проходящего через указанные города ровно по одному разу с последующим возвратом в исходный город.

Динамическое программирование по подмножествам.

Задача о коммивояжере представляет собой поиск кратчайшего гамильтонова цикла в графе. Зафиксируем начальную вершину s и будем искать гамильтонов цикл наименьшей стоимости — путь от s до s , проходящий по всем вершинам (кроме первоначальной) один раз. Т.к. искомый цикл проходит через каждую вершину, то выбор s не имеет значения. Поэтому будем считать $s = 0$.

Подмножества вершин будем кодировать битовыми векторами, обозначим $mask_i$ значение i -ого бита в векторе $mask$.

Обозначим $d[i][mask]$ как наименьшую стоимость пути из вершины i в вершину 0, проходящую (не считая вершины i) единожды по всем тем и только тем вершинам j , для которых $mask_j = 1$ (т.е. $d[i][mask]$ уже найденный оптимальный путь от i -ой вершины до 0-ой, проходящий через те вершины, где $mask_j = 1$. Если $mask_j = 0$, то эти вершины еще не посещены).

Алгоритм поиска цикла будет выглядеть следующим образом:

- Начальное состояние — когда находимся в 0-й вершине, ни одна вершина не посещена, а пройденный путь равен 0 (т.е. $i = 0$ и $mask = 0$).

- Для остальных состояний ($i \neq 0$ или $mask \neq 0$) перебираем все возможные переходы в i -ую вершину из любой посещенной ранее и выбираем минимальный результат.
- Если возможные переходы отсутствуют, решения для данной подзадачи не существует (обозначим ответ для такой подзадачи как “no path”).

Алгоритм лучшего шага

АЛШ – алгоритм лучшего шага. Похож на АБС, так же идёт только вперёд, но на каждом шаге добавляет дугу не с минимальным весом (s), а с минимальным значением ($s + L$), где L – нижняя оценка стоимости остатка решения. АЛШ-1: L считается на основе полусуммы легчайших рёбер (берётся вес легчайшей допустимой дуги, входящей в начальную вершину куска, и вес легчайшей допустимой дуги, исходящей из конечной вершины куска).

Выполнение.

Для реализации алгоритма Хелда-Карпа рекурсивным методом была написана функция *held_karp*, принимающая на вход матрицу весов.

S – битовая маска, v – текущая вершина. Для каждой пары S, v словарь *cost_cache* хранит минимальную стоимость пути из вершины 0 до вершины v , посетив ровно вершины из маски S . Для восстановления пути словарь *parent* хранит предыдущую вершину для каждой такой пары.

Вложенная функция *min_cost* вычисляет и возвращает минимальную стоимость для пары S, v . Базовый случай: $S == 1, v == 0$ – стоим в начальной вершине, стоимость 0. Если пара уже есть в *cost_cache* – возвращаем сохраненное значение. Если пары нет в *cost_cache*, то перебираем все вершины $u \in S$ как возможные предыдущие, рекурсивно вызывая *min_cost* для пары S без v, u и выбираем минимум. Результат сохраняется в *cost_cache*, предыдущая вершина в *parent*.

После вычисления перебираем все возможные последние вершины v перед возвратом в 0 и выбирается та, через которую полный маршрут минимален. Если минимальная стоимость осталась INF , то возвращается “no path”. Если путь существует, то он восстанавливается в обратном порядке с помощью *parent*. Начиная с конца, убираем последнюю вершину из маски и переходим к предыдущей.

Функция возвращает пару: минимальную стоимость маршрута и список вершин такого маршрута, начинающийся и заканчивающийся 0.

Временная и пространственная сложность: $O(2^n \cdot n)$

Для реализации алгоритма лучшего шага АЛШ-1 была написана функция *alsh1*.

На каждом шаге вершина для перехода выбирается как минимум $s + L$, где s – вес ребра до кандидата, L – нижняя оценка стоимости остатка пути.

Функции *min_incoming* и *min_outgoing* возвращают минимальный вес ребра, входящего в вершину v среди непосещенных и минимальный вес ребра, выходящих из v . Функция *compute_L* вычисляет нижнюю оценку стоимости остатка пути как полусумму легчайших допустимых рёбер по всем кускам. Для каждого куска берется минимальная допустимая дуга, входящая в начало куска и минимальная допустимая дуга, исходящая из конца куска.

В основном цикле происходит $n - 1$ шагов. На каждом шаге для каждого кандидата считается $s + L$ и выбирается лучший. Если на каком-то шаге кандидата не нашлось, то возвращается “no path”. После обхода всех вершин проверяется, можно ли вернуться в 0, если нет, то возвращается “no path”. Функция возвращает пару: стоимость маршрута и список вершин пути, начинающий и заканчивающийся 0.

Временная сложность: $O(n^3)$ – внешний цикл n шагов, внутренний перебор n кандидатов, для каждого *compute_L* обход n вершин

Пространственная сложность $O(n)$ – хранится *visited* и *path*

Исходный код программы см. в Приложении А.

Результаты тестирования см. в Приложении Б.

Вывод

Были изучены и реализованы алгоритмы для решения задачи коммивояжёра. Точный алгоритм – рекурсивный алгоритм Херда-Карпа, приближенный – алгоритм лучшего шага с оценкой L . Точный алгоритм работает дольше, но приближенный не всегда дает верное решение.

ПРИЛОЖЕНИЕ А.

ИСХОДНЫЙ КОД ПРОГРАММЫ.

Название файла: generate_matrix.py

```
import random

def generate_matrix(n, symmetric=True):
    matrix = [[0] * n for x in range(n)]
    for i in range(n):
        for j in range(i + 1, n):
            weight = random.randint(1, 100)
            matrix[i][j] = weight
            if symmetric:
                matrix[j][i] = weight
            else:
                matrix[j][i] = random.randint(1, 100)
    return matrix

def save_matrix(matrix, file_name="matrix.txt"):
    n = len(matrix)
    with open(file_name, 'w') as f:
        for row in matrix:
            f.write(" ".join(map(str, row)) + "\n")

def generate_and_save(n, symmetric=False, file_name="matrix.txt"):
    matrix = generate_matrix(n, symmetric)
    save_matrix(matrix, file_name)
    return matrix
```

Название файла: held_karp.py

```
import generate_matrix

def held_karp(matrix):
    n = len(matrix)
    INF = float('inf')
    cost_cache = {}
    def min_cost(S, v):
        if S == 1 and v == 0:
            return 0
        if (S, v) in cost_cache:
            return cost_cache[(S, v)]
        result = INF
        for u in range(n):
            if not (S >> u & 1) or u == v or matrix[u][v] == 0:
                continue
            cost = min_cost(S ^ (1 << v), u) + matrix[u][v]
```

```

        if cost < result:
            result = cost
            parent[(S, v)] = u
        cost_cache[(S, v)] = result
    return result

parent = {}
full = (1 << n) - 1
best_cost = INF
last = -1
for v in range(1, n):
    if matrix[v][0] == 0:
        continue
    cost = min_cost(full, v) + matrix[v][0]
    if cost < best_cost:
        best_cost = cost
        last = v
if best_cost == INF:
    return INF, "no path"
path = []
S = full
v = last
while v != -1:
    path.append(v)
    prev = parent.get((S, v), -1)
    S = S ^ (1 << v)
    v = prev
path.reverse()
path.append(0)

return best_cost, path

```

```

def console_input():
    n = int(input())
    matrix = []
    for i in range(n):
        row = list(map(int, input().split()))
        matrix.append(row)
    min_cost, path = held_karp(matrix)

```



```

if min_cost < float('inf'):
    print(min_cost, ' '.join(map(str, path)), sep='\n')
else:
    print(path)

```

```

def file_input():
    generate_matrix.generate_and_save(5)
    f = open("./matrix.txt", 'r')
    matrix = []
    for line in f:
        row = list(map(int, line.split()))
        matrix.append(row)
    for i in range(len(matrix)):
        print(' '.join(map(str, matrix[i])))
    min_cost, path = held_karp(matrix)
    if min_cost < float('inf'):
        print(min_cost, ' '.join(map(str, path)), sep='\n')
    else:
        print(path)

```

file_input()

#console_input()

Название файла: alsh1.py

import generate_matrix

```

def alsh1(matrix):
    n = len(matrix)
    INF = float('inf')
    def min_incoming(v):
        result = INF
        for i in range(n):
            if i != v and not visited[i] and matrix[i][v] != 0:
                if matrix[i][v] < result:
                    result = matrix[i][v]
        return result

    def min_outgoing(v):
        result = INF
        for j in range(n):
            if j != v and not visited[j] and matrix[v][j] != 0:
                if matrix[v][j] < result:
                    result = matrix[v][j]
        return result

    def compute_L():

```

```

total = 0.0
for v in range(n):
    if not visited[v]:
        inc = min_incoming(v)
        out = min_outgoing(v)
        if inc == INF or out == INF:
            continue
        total += inc + out
inc = min_incoming(path[0])
out = min_outgoing(path[-1])
if inc == INF or out == INF:
    return total / 2.0
total += inc + out
return total / 2.0

visited = [0] * n
path = [0]
visited[0] = 1
best_cost = 0
for i in range(n - 1):
    best_next = -1
    best_value = INF
    for candidate in range(n):
        if visited[candidate] or matrix[path[-1]][candidate] == 0:
            continue
        s = matrix[path[-1]][candidate]
        visited[candidate] = 1
        path.append(candidate)
        L = compute_L()
        path.pop()
        visited[candidate] = 0
        if s + L < best_value:
            best_value = s + L
            best_next = candidate
    if best_next == -1:
        return INF, "no path"
    best_cost += matrix[path[-1]][best_next]
    visited[best_next] = 1
    path.append(best_next)
if matrix[path[-1]][0] == 0:
    return INF, "no path"
best_cost += matrix[path[-1]][0]
path.append(0)
return best_cost, path

```

```

def console_input():
    n = int(input())
    matrix = []
    for i in range(n):
        row = list(map(int, input().split()))
        matrix.append(row)
    min_cost, path = alsh1(matrix)
    if min_cost < float('inf'):
        print(min_cost, ' '.join(map(str, path)), sep='\n')
    else:
        print(path)

```

```
def file_input():
    generate_matrix.generate_and_save(5)
    f = open("./matrix.txt", 'r')
    matrix = []
    for line in f:
        row = list(map(int, line.split()))
        matrix.append(row)
    for i in range(len(matrix)):
        print(' '.join(map(str, matrix[i])))
    min_cost, path = alsh1(matrix)
    if min_cost < float('inf'):
        print(min_cost, ' '.join(map(str, path)), sep='\n')
    else:
        print(path)

file_input()
```

ПРИЛОЖЕНИЕ Б.
ТЕСТИРОВАНИЕ.

№	Входные данные	Точный метод	Приближенный метод	Комментарии
1	5 0 1 13 23 7 12 0 15 18 28 21 29 0 33 28 23 19 34 0 38 5 40 7 39 0	78 0 4 2 3 1 0	78 0 4 2 3 1 0	Одинаковый результат на примере из задания.
2	5 0 30 65 78 4 72 0 26 92 84 90 70 0 54 29 58 76 36 0 1 98 21 90 55 0	163 0 4 1 2 3 0	198 0 1 2 4 3 0	Жадный алгоритм делает неоптимальный выбор на первом шаге и получает неверное итоговое решение.